

Home Components Software Backend NodeJS components

Darwin NodeJs Microservice Journey

Introduction

The purpose of this documentation is to provide a **step-by-step guide** on how to orchestrate the integration of microservices built with the **Darwin NodeJS** framework within the GLUON platform.

This guide will allow you to understand how to build and deploy our microservices through a CI/CD process.

All deployments will be done in a Kubernetes cluster from an immutable image that we will previously upload to a registry (Harbor/JFROG/ECR).

Setup your local environment

Before start with the creation of the components in Gluon, maybe you need to configure your local environment. In the next links you can find the steps to configure your environment and build your front apps in your local machine.

- [Install node](#)
- [Configuring npm](#)

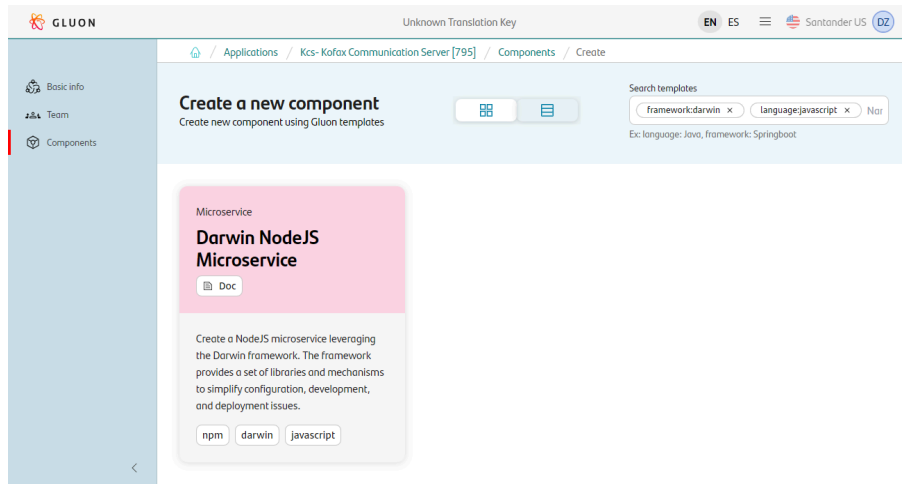
Create Component

Gluon Portal

First you have to **onboard your application**. Once you have your application created, you can start creating your component.

To create a component, follow the steps described in **Component Management**, searching for the component to be created.

You have to select the type of component you want to create, in this case you are going to create a **Darwin NodeJS Microservice**.

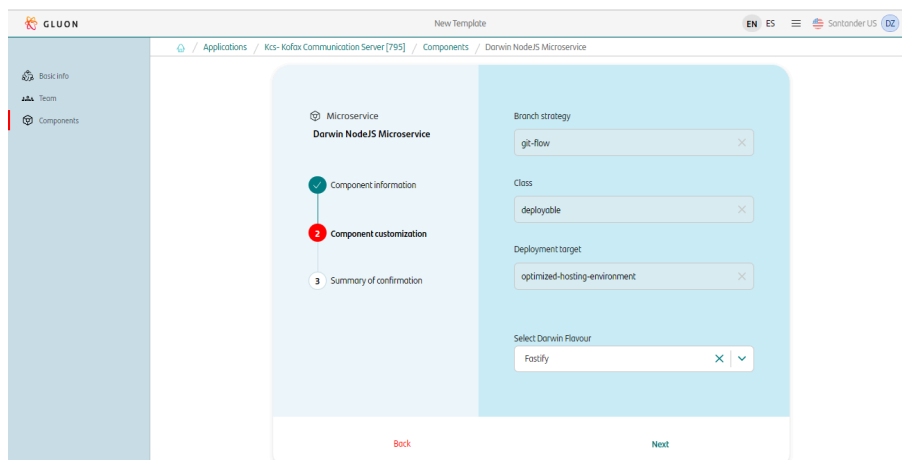


Remember



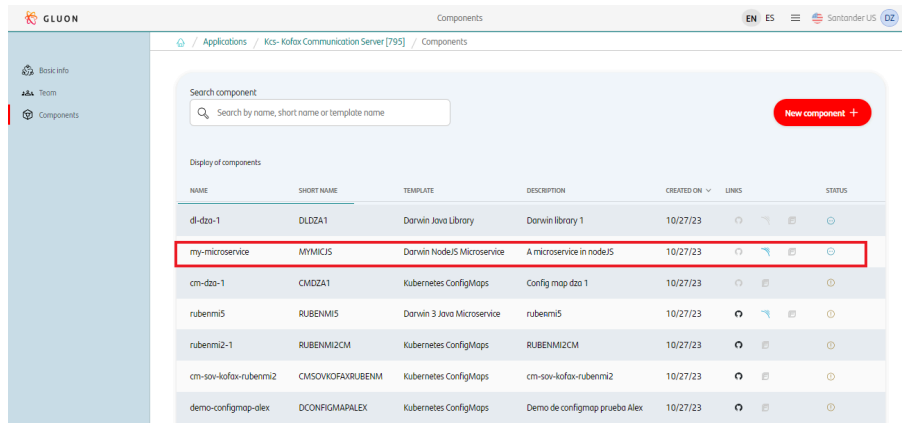
To follow the naming convention visit [Repository Naming Convention](#).

The user can customize the type of application that they want to create. For this example we have created a Darwin microservice with the following characteristics:



- **Branch Strategy:** git-flow
- **Class:** deployable
- **Deployment target:** optimized-hosting-environment
- **Darwin Flavour:** Fastify

Once the component is created we can see under the application that there is a new repository created with the name of the component, Sonar project and Fortify project.



We have the following links in:

Item	Link	Role Permission
GitHub Repository	Link to the repository created in GitHub	Developer (RW) /Technical Lead (Maintain) All roles explained
Sonar Project	Link to the Sonar project created	All Users (Read)
Fortify Project	Link to the Fortify project created	All Users (Read)

Darwin Microservice Template

Branches

Gluon works with two branches that will need to be incorporated into our project:

- The main branch (main by default or master in old projects)
- The integration branch (development/develop by default)

This is because the workflows works with GitFlow strategy, so we need to have the integration branch to merge the feature branches and create the Pull Request to the main branch.

Trunk Based Development

Here, Gluon works with only one branch that will need to be incorporated into our project:

- The main branch (main by default or master in old projects)

With trunk-based development, we will merge feature branches with Pull Requests directly into the main branch.

Structure



Warning



The structure of the Darwin NodeJS microservice is documented on the `Readme.md` and depends on the type of microservice that you are using (Fastify or GraphQL).

```
.
├── .editorconfig      # EditorConfig style defined
├── .eslintignore      # Configuration for linter
├── .eslintrc.js       # Configuration for linter
├── .gitignore         # Files to ignore in git
├── .npmrc             # NPM configuration file
├── .npmignore         # Files to ignore in git
├── .github/           # Contains all github security and
                        # quality workflows
├── .gluon/            # Contains all github deployment
                        # workflows and helm chart values
├── .husky/            # Contains pre-commit hook code
├── api
│   └── swagger        # API Documentation
├── app/              # Source code of application
│   ├── config/       # Folder to store configurations,
                        # variables, etc.
│   ├── controllers/  # Folder that contains the application
                        # controllers.
│   ├── routes/       # Routes of the application
│   │   └── tools/    # Folder that contains Joi validation
│   │       └── validation/ # Folder that contains Joi schemas.
│   └── server/       # Define server-side startup point
├── src/              # Source code of application, similar
                        # to app/ (typescript and GraphQL only)
├── Dockerfile        # Scripts necessary to deploy the
                        # component
├── catalog.properties # Cataloguing properties
├── package-lock.json  # Automatically generated for any
                        # operations where npm
│                       # modifies either the node_modules
                        # tree, or package.json
├── package.json      # Application dependencies and
                        # configuration
├── changelog         # Changelog example
├── readme.md         # Readme example
└── test              # Application tests
```

For more information about the utilities and libraries of Darwin microservices, please refer to the [Darwin Node Microservice Archetype documentation](#) provided by the framework.

Local Running

Cloning your repository

Once we have the device properly configured, the first step is to clone the project locally.

To do so, visit [How to clone de project](#).

Run your microservice

To Run the microservice, first you will have to install the dependencies. To do so, run the following command:

```
npm install
```



After that, you can run the microservice with:

```
npm run start
```



If the microservice is in typescript, the previous command will fail unless the following command is executed:

```
npm run build
```



Infrastructure

You have to request the following infrastructure resources.

Registry

- Project: You need a project in the registry (**Harbor** , **JFrog** or **ECR**) to upload your image.
- Credentials: **user/password** with privileges to upload images to the project.

Kubernetes

- **Namespace:** You need a **namespace** in the **Kubernetes cluster** to deploy your microservice (and other kubernetes components) in the CERT, PRE and PRO environments. You can create your namespace in Gluon following [this guide](#)
- **Credentials:** **Service Account** with the credentials to deploy in the namespace.

Configuration

- **Configuration Configmap:** By default, the microservice does not require a configuration file, but it's also possible to create a configmap in the namespace with your **external environment-dependent microservice configuration**. You can create the configmap component in Gluon following [this guide](#)
- **Intern Configmap:** it's also possible to create a configmap in the namespace with **internationalization values for your microservice**. You can create the configmap component following [this same guide](#).



Configmap



- **The input parameters required to configure using a configmap vary based on the application.** Visit [Helm documentation](#) to know more about.
- Now the **name of the configmap is customisable**. You can use any name for the secret and configmap. The name of the secret and configmap can be overwritten in the `.gluon/cd/values.yaml` file.



Example of the configmap with environment variables



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-[microservice-name] or cm-[microservice-
applicationName]
  namespace: [namespace]
data:
  DARWIN_TEST_VALUE: YWRtaW4=
  DARWIN_TEST_VALUE2: MWYyZDF1MmU2N2Rm
```



Deployment Configuration

The company used to create the components must have been provided the **Gluon Open Application Model**. The deployment model uses the `oam-application-definition.yaml` file to manage infrastructure and registries in a standardized way across different platforms.

OAM Configuration

To deploy a Gluon Component, the parameters of the target infrastructure must be configured in the **Gluon Application Model** component of the technical application, **configuring the data** necessary depending on the type of Kubernetes Cluster used.

Configure environments

By default, the component is created with the folders `cert`, `pre`, and `pro`, which cover the most common use case. Each of these folders corresponds to an environment in which you want to deploy the Gluon Component. If, for example, you want to deploy in two production environments called `pre-pro` and `back-pro`, the folder structure that should be created in the `cd` folder is `dev`, `pre`, `pre-pro`, and `back-pro`, each with the `cd.yaml` file where the infrastructure to be deployed must be configured.

For it to work correctly, the name of the folder must exactly match the name of the "name" property (in the example shown below, it would be the `cert` value) of the `oam-application-definition.yml` file of the Gluon Application Model component.

So, to define a deployment environments, it is necessary to describe the `name` and `type` fields:

- `name` : The name of the environment. Each Application could give a different name to the environments. It must be unique in the `oam` file.
- `type` : The type of the environment. The values allowed are `certification`, `preproduction`, and `production`.

Below is an example of the configuration of a certification environment, called `cert`, where one infrastructure have been configured to deploy for Amazon Elastic-Kubernetes Service Cluster. Many properties have been omitted for this example, but for it to work correctly, the rest of the mandatory data must be configured.

oam-application-definition.yml



```

environments:
  - name: cert
    type: certification
    infrastructures:
      - id: CI000000000001
        properties:
          type: KUBERNETES
          apiServer: https://12345asdfg67890hjk1.sk1.eu-west-
1.eks.amazonaws.com
          namespace: gluon-back-java
          credentialUserId: AWS_ACCESS_KEY_ID
          credentialPassId: AWS_SECRET_ACCESS_KEY_ID
          provider: eks
          cloud: aws
          account: 123456789012
          role: AccountAutomationNonPro
          region: eu-west-1
          clusterName: sgt1a1reksgluondeksd111
          artifact-store: CI000000000002
          (...)

      - id: CI000000000002
        type: ARTIFACT-STORE
        properties:
          type: ecr
          registry: 12345asdfg67890hjk1.dkr.ecr.eu-west-
1.amazonaws.com
          project-path: project-anme
          usernameId: AWS_ACCESS_KEY_ID
          passwordId: AWS_SECRET_ACCESS_KEY_ID
          role: AccountAutomationNonPro
          snapshots: true
          (...)

```



Cluster Authentication

There are three ways to authenticate against a Cluster:

- **credentialsId**: Use property credentialsId setting the name of the github secret storing the token. **This is the recommended method.**
- **credentialUserId / credentialPassId**: Use properties credentialUserId/credentialPassId placing there github secret names with username and password to use to authenticate with server.
- Verify that the user have the necessary permissions to deploy in the cluster and in the artifact store.
- **Using Hashicorp Vault**: The system will automatically search in the Hashicorp Vault for the deployment secret

The next step is to configure the `cd.yml` file, where you can set up the infrastructures where the Gluon Microservice will be deployed for that environment. For each one, the following properties must be configured:

Property	Description	Example
ci_id	Identifier of the infrastructure in the oam-application-definition.yml file	CI00000000097
configurationFiles	Path to the Helm configuration file in that environment	.gluon/cd/cert/values-cert.yml

Following the previous example from the oam-application-definition.yml file, the content of the `cd.yml` file to deploy on Amazon Elastic-Kubernetes Service (EKS) and the container images on Amazon Elastic Container Registry (ECR) would be as follows:

cd.yml

```
- ci_id: CI00000000001
  configurationFiles:
    - .gluon/cd/cert/values-cert.yml
    - .gluon/cd/values.yml
```

!!IMPORTANT - Considerations:

- The workflow will deploy on all infrastructures that are included in the cd.yml file of the environment in which it is going to be executed.
- The infrastructures included in the cd.yml file must be registered in the environment in the oam-application-definition.yml file.
- These values indicate where the microservice will be deployed. They do not depend on **GLUON**. They are specific to the application to be deployed.

For getting to know how to configure the Gluon Open Application Model repository associated with the company where the component is generated, the following documentation is available:

- [How to configure the Gluon Open Application Model \(OAM\)](#)
- [All the parameters available by type of infrastructure component](#)

- [Kubernetes deployment examples for OAM Configuration](#)

Component Configuration

Configuration Files

- **Dockerfile**: Configuration to build the image
- **./gluon/ci/properties.env**: Properties with the CI/CD configuration
- **./gluon/cd/values.yaml**: Configuration with the values to deploy the image.
- **./gluon/cd/[ENVIRONMENT]/cd.yaml**: Reference to the OAM configuration for any [ENVIRONMENT].
- **./gluon/cd/[ENVIRONMENT]/values-[ENVIRONMENT].yaml**: specific deployment values for any [ENVIRONMENT].

Properties

The **properties.env** file contains the configuration for the CI/CD pipeline. It is generated automatically. The parameters that are configured by default are:

Variable	Required	Description
SONAR_PROJECT_KEY	true	Project key in Sonar
FORTIFY_PROJECT	true	Project name in Fortify
JAVA_VERSION	true	Java version to use
ARTIFACT_NATIVE_COMPILATION	false	Artifact native compilation

Naming convention

The project name in **Sonar** and **Fortify** must be the same as the component repository name. To avoid possible errors, the **properties.env** file is generated with the necessary values for the variables

SONAR_PROJECT_KEY and **FORTIFY_PROJECT**.

Example values:

SONAR_PROJECT_KEY="sgt-gluonad-probemicroframework"

FORTIFY_PROJECT="sgt-gluonad-probemicroframework"

All the properties

See common properties [here](#)

Dockerfile

This file allow us to build the image with the microservice in the *Build* stage of the CI/CD pipeline.

One example for Dockerfile is:

Dockerfile Proposal

```
1 FROM
2 registry.global.ccc.srvb.bo.paas.cloudcenter.corp/produb
3 an/nodejs-16-ubi8:1.0.0.RELEASE
4
5 ARG ARTIFACT_PATH
6
7 USER root
8 ADD $ARTIFACT_PATH /tmp/app.zip
9 RUN npm config delete proxy && \
10     npm config set strict-ssl false && \
11     curl -k
12 https://nexus.alm.europe.cloudcenter.corp/repository/alm
13 mc-san-darwin-sources-raw-
14 releases/nodejs/certs/1.0.0/SANTANDER_CA_CERTS.zip -o
15 /tmp/san-cert.zip && \
16     unzip -o /tmp/san-cert.zip -d /opt/app/ && \
17     unzip -o /tmp/app.zip -d /opt/app
18 ENV NODE_EXTRA_CA_CERTS /opt/app/SANTANDER_CA_CERTS.pem
19 WORKDIR /opt/app
20 RUN npm install --production --verbose
21 USER 23550
22 WORKDIR /opt/produban
```

```
ENTRYPOINT [ "./control.sh" ]  
CMD [ "start" ]
```



Registry Configuration

These values indicate where the image of the microservice will be deployed. They do not depend on GLUON. They are specific to the application to be deployed.

Deployment Environment



Credentials and Secrets



Make sure the parameters set up as secrets in your OAM repository are also set up in your microservice repository.

Secrets can be set up, managed and deployed with **the vault**.

In that set of files, we are going to define the necessary infrastructure references so that the helm chart can deploy the configmap correctly in the configured environment. The Continuous Deployment file (`cd.yml`) must contain the target deployment configuration that we want to use for deploying the component. For each environment (cert, pre, pro), we have a folder with the `cd.yml` file, and there, we can define several infrastructures to deploy in as many regions as we need. Remember that `cd.yml` files are empty, and the developer is responsible for filling them with the necessary deployment information.

```
.gluon  
├── cd  
│   ├── cert  
│   │   └── cd.yml  
│   ├── pre  
│   │   └── cd.yml  
│   └── pro  
│       └── cd.yml
```

For that purpose, it will only be necessary to add the `ci_id` identifiers defined by environment type in the `oam-application-definition.yml` file inside **Gluon Open Application Model repository** associated with the company of the component. Keep in mind that **`ci_id` must be the same as we have in OAM the config file**. The `configuration_files` key allows setting the `values` chart files that they are necessary to be able to deploy in the infrastructures to which they refer.

Property	Description	Example
ci_id	Identifier of the infrastructure in the oam-application-definition.yml file	CI00000000097
configurationFiles	Path to the Helm configuration file in that environment	.gluon/cd/cert/values-cert.yml

The following template shows an example of the structure that the `cd.yml` file should have:

```
# Kubernetes cluster in AWS
- ci_id: CI00000000001
  configuration_files:
    - .gluon/cd/values.yml
    - .gluon/cd/{environment}/values-{environment}.yml
# Kubernetes cluster in Azure
- ci_id: CI00000000002
  configuration_files:
    - .gluon/cd/values.yml
    - .gluon/cd/{environment}/values-{environment}.yml
```

See the full list of examples of how to set up your deployment infrastructure [here](#)

For getting more information about how-to-configure the deployment environment files, please refer to the [Continuous Deployment file documentation](#).

For getting to know how to configure the `Gluon Open Application Model` repository, the following documentation is available [here](#)



Blue & Green deployment



This functionality is currently only available if deployed on infrastructure in Spain.

More information about [blue & green deployment](#)

Helm Configuration

You must have the next files in the `.gluon/cd` folder of your project with the following structure:

```

.gluon
├── cd
│   ├── cert
│   │   └── values-cert.yml
│   ├── pre
│   │   └── values-pre.yml
│   ├── pro
│   │   └── values-pro.yml
│   └── values.yml

```

Description of the files:

- **values.yml**: Helm values file with the default values to deploy the microservice.
- **values-cert.yml**: Helm values file with the values to deploy the microservice in the CERT environment.
- **values-pre.yml**: Helm values file with the values to deploy the microservice in the PRE environment.
- **values-pro.yml**: Helm values file with the values to deploy the microservice in the PRO environment.

Examples of values files:

values.yml

```

1  # Default values for darwin micro-node chart.
2  # This is a YAML-formatted file.
3  # Declare variables to be passed into your templates.
4  ## Active
5  active: online
6  #####
7  # Labels and Annotations #
8  #####
9  ## @param commonLabels [object] Labels to add to all
10 deployed objects
11 ##
12 commonLabels: {}
13 ## @param commonAnnotations [object] Annotations to add
14 to all deployed objects
15 ##
16 commonAnnotations: {}
17 ## @param podLabels [object] Extra labels for the micro
18 service pods
19 ## Ref:
20 https://kubernetes.io/docs/concepts/overview/working-
21 with-objects/labels/
22 ##

```

```

23 podLabels: {}
24 ## @param podAnnotations [object] Annotations for the
25 micro service pods
26 ## ref:
27 https://kubernetes.io/docs/concepts/overview/working-
28 with-objects/annotations/
29 ##
30 podAnnotations: {}
31 ## Darwin Micro Service image version
32 ## @param image.registry darwin micro image registry
33 ## @param image.repository darwin micro image name
34 ## @param image.tag darwin micro image tag
35 ## @param image.digest darwin micro image digest in the
36 way sha256:aa.... Please note this parameter, if set,
37 will override the tag
38 ##
39 image:
40   registry:
41   repository:
42   tag:
43   digest: ""
44   ## @param image.pullPolicy darwin micro image pull
45 policy
46   ## Specify a imagePullPolicy
47   ## Defaults to 'Always' if image tag is 'latest',
48 else set to 'IfNotPresent'
49   ## ref: https://kubernetes.io/docs/user-
50 guide/images/#pre-pulling-images
51   ##
52   pullPolicy: IfNotPresent
53 ## @param replicaCount Number of replicas of the micro
54 service pod
55 ##
56 replicaCount: 1
57 ## @param lifecycle [object] Override default micro
58 service container hooks
59 ## If you do not set this property, the default hook
60 is: `{lifecycle: {preStop: {exec: {command: [sleep
61 120]}}}}`
62 ##
63 lifecycle: {}
64
65 ## @param terminationGracePeriodSeconds In seconds,
66 time the given to the pod needs to terminate gracefully
67 ## ref:
68 https://kubernetes.io/docs/concepts/workloads/pods/pod/
69 #termination-of-pods
70 ##
71 terminationGracePeriodSeconds: 300
72
73 ## Darwin Micro Service pods' Security Context
74 ## ref: https://kubernetes.io/docs/tasks/configure-pod-
75 container/security-context/#set-the-security-context-
76 for-a-pod
77 ## @param podSecurityContext.enabled Enable security
78 context for the pods
79 ## @param podSecurityContext.fsGroup Set Darwin Micro

```

```

80 Service pod's Security Context fsGroup
81 ## e.g:
82 ##   podSecurityContext:
83 ##     enabled: true
84 ##     fsGroup: 1001
85 ##
86 podSecurityContext:
87   enabled: false
88
89 ## Darwin Micro Service containers' Security Context
90 ## ref: https://kubernetes.io/docs/tasks/configure-pod-
91 container/security-context/#set-the-security-context-
92 for-a-container
93 ## @param containerSecurityContext.enabled Enable
94 Darwin Micro Service containers' Security Context
95 ## @param containerSecurityContext.runAsUser Set Darwin
96 Micro Service containers' Security Context runAsUser
97 ## @param containerSecurityContext.runAsNonRoot Set
98 Darwin Micro Service containers' Security Context
99 runAsNonRoot
100 ## @param
101 containerSecurityContext.allowPrivilegeEscalation Force
102 the child process to be run as nonprivilege
103 ## e.g:
104 ##   containerSecurityContext:
105 ##     enabled: true
106 ##     runAsUser: 1001
107 ##     runAsNonRoot: true
108 ##     allowPrivilegeEscalation: false
109 ##
110 containerSecurityContext:
111   enabled: false.
112
113 ## @param podNodeSelector.enabled Enable node selection
114 for the pods
115 ## @param podNodeSelector.{labelName1} Node label name1
116 and value1
117 ## @param podNodeSelector.{labelName2} Node label name2
118 and value2
119 ## e.g:
120 ##   podNodeSelector:
121 ##     enabled: true
122 ##     label1: value1
123 ##     label2: value2
124 ##     label3: value3
125 ##
126 podNodeSelector:
127   enabled: false
128
129 #####
130 # Port #
131 #####
132 ## Configures the ports microservice listens on
133 ## @param containerPorts.http Sets http port inside
134 container
135 ## @param containerPorts.https Sets https port inside
136 container

```

```

137  ##
138  containerPorts:
139    http: 8080
140    https: 8443
141
142  ## @param extraContainerPorts Array of additional
143  container ports for the container
144  ## e.g:
145  ## extraContainerPorts:
146  ##   - name: grpc
147  ##     containerPort: 4317
148  ##
149  extraContainerPorts: []
150
151
152  #####
153  # Resources #
154  #####
155  resources:
156    limits:
157      memory: 1G
158      cpu: 500m
159    requests:
160      memory: 512M
161      cpu: 100m
162
163  ## @param updateStrategy.rollingUpdate deployment
164  rolling update configuration parameters
165  ## ref:
166  https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#strategy
167  ## Note: Set to Recreate if you use persistent volume
168  that cannot be mounted by more than one pods to make
169  sure the pods is destroyed first.
170  ## E.g:
171  ## updateStrategy:
172  ##   type: RollingUpdate
173  ##   rollingUpdate:
174  ##     maxSurge: 25%
175  ##     maxUnavailable: 25%
176  ##
177  ##
178  updateStrategy:
179    type: RollingUpdate
180    rollingUpdate: {}
181
182  #####
183  # Liveness Probe #
184  #####
185  livenessProbe:
186    enabled: true
187    httpGet:
188      path: /health
189    initialDelaySeconds: 30
190    timeoutSeconds: 1
191    periodSeconds: 10
192    successThreshold: 1
193    failureThreshold: 3

```

```

194
195 #####
196 # Readiness Probe #
197 #####
198 readinessProbe:
199   enabled: true
200   httpGet:
201     path: /health
202   initialDelaySeconds: 10
203   timeoutSeconds: 1
204   periodSeconds: 10
205   successThreshold: 1
206   failureThreshold: 3
207
208 ## @param controller.initContainers Add additional init
209 containers to the Controller pods
210 initContainers: []
211
212 #####
213 # Service parameters #
214 #####
215 service:
216   ## @param service.type Kubernetes service type
217   type: ClusterIP
218   ## @param service.overrideServiceName Override the
219 default service name
220   overrideServiceName:
221   ## @param service.ports.http Service HTTP port
222   ## @param service.ports.https Service HTTPS port
223   ##
224   ports:
225     http: 8080
226     https: 8443
227   targetPort:
228     http: 8080
229     https: 8443
230   ## @param service.annotations [object] Additional
231 annotations for the service of the micro
232   ##
233   annotations: {}
234
235 #####
236 # Environment variables #
237 #####
238
239 ## @param extraEnvVars Extra environment variables to
240 be set on Node container
241 ## For example:
242 ## - name: BEARER_AUTH
243 ##   value: true
244 ## - name: DARWIN_CM_KEY
245 ##   valueFrom:
246 ##     configMapKeyRef:
247 ##       name: "cm4"
248 ##       key: LOGGING_ROOT_LEVEL
249 ##     optional: false
250 ## - name: DARWIN_SECRET_KEY

```

```

251  ##      valueFrom:
252  ##      secretKeyRef:
253  ##          name: "datagrid-app"
254  ##          key: "application-password"
255  ##          optional: false
256  ##
257  extraEnvVars: []
258
259  ## @param extraEnvVarsCM Name of existing ConfigMap
260  ## containing extra environment variables
261  ##
262  extraEnvVarsCM:
263
264  ## @param extraEnvVarsSecret Name of existing Secret
265  ## containing extra environment variables
266  ##
267  extraEnvVarsSecret:
268
269  ## @section Volumes ##
270
271  ## @param extraVolumes Optionally specify extra list of
272  ## additional volume for the container(s)
273  extraVolumes: []
274
275  ## @param extraVolumeMounts Optionally specify extra
276  ## list of additional volumeMounts for the container(s)
277  extraVolumeMounts: []
278
279  ## TLS configuration
280  ##
281  ssl:
282  ## @param ssl.enabled Enable init container that
283  ## generate the truststore and keystore
284  ##
285  enabled: false
286
287  #####
288  # Darwin configuration #
289  #####
290  darwin:
291  ## @param darwin.version. Darwin Framework Version.
292  ## Examples values: `4.0.1`, `3.1.0`
293  version:
294  ## @param darwin.gitRepo. Git repository with the
295  ## source code of the microservice.
296  ## Example value:
297  ## `https://github.alm.europe.cloudcenter.corp/sanes-
298  ## darwin-poc/poc-certificates.git`
299  gitRepo:
300  ## @param darwin.technologyVersion. Node version of
301  ## the image.
302  technologyVersion:
303  ## @param darwin.region. Cluster identifier.
304  ## Possible values: `""`, `bo1`, `bo2`, `weu1.az` or
305  ## `weu2.az`
306  region: ""
307  ## @param darwin.suffix. Suffix to blue green

```

```

deployments.
  ## Possible values: `""`, `-b` or `-g`
  suffix: ""
  logging_level_root: INFO
  tz: Europe/Madrid
  ## Configuration
  ##
  ## @param darwin.config Allows to customize the
  configuration files.
  config:
    ## @param darwin.configType Allows to set the
    configuration from ConfigMap or Configuration Service
    ## It indicates the location from which the
    configuration is retrieved.
    ## Four possible values are allowed:
    ## - "cm" links a Kubernetes ConfigMap with the
    name cm-{app_name}. It will be mounted.
    ## - "secret" links a Kubernetes Secret with the
    name secret-{app_name}. It will be mounted.
    ## - "cm-secret" links a Kubernetes ConfigMap with
    name cm-{app_name} and a Kubernetes Secret with name
    secret-{app_name}. They will be mounted.
    ## - "none" does not link any Kubernetes ConfigMap
    or Secret.
    type: none
    ## @param darwin.configuration.overrideCmName
    Override the default ConfigMap name
    overrideConfigMapName:
    ## @param darwin.configuration.overrideSecretName
    Override the default Secret name
    overrideSecretName:
    ## Internationalization
    ##
    ## @param darwin.i18n Allows to customize the
    internationalization files.
    i18n:
    ## @param i18n.enabled Allow customizing the
    internationalization files.
    ##
    enabled: false
    ## @param i18n.path Define the path to store the
    literal and internationalization files, if is filled.
    ## If it is not filled, the default path is
    '/etc/i18n/locales'.
    path:
    ## @param i18n.overrideConfigMapName Allows to
    override the name of the configuration Map that
    contains the files with the translations.
    ## if it is not filled, the default name is i18n-
    {app_name}.
    overrideConfigMapName:
    node:
    env: DEV
    cloud_config:
    enable: true
    failfast: true
    # Values to connect with Configuration Service

```

```

    uri: http://configuration-service:8080
    retry:
      initialInterval : 3000
      maxInterval : 6000
      maxAttempts : 10
    # Values for security module
    security:
      pkc:
        uri:
https://srvnuarintra.santander.dev.corp/pkc/v1/publicKey/
      # Values for logger module
      logger:
        kafka:
          host:
"sanlckfbrd0008.santander.dev.corp:9094,sanlckfbrd0009.santander.dev.corp:9094,sanlckfbrd0010.santander.dev.corp:9094,sanlckfbrd0011.santander.dev.corp:9094"

```

values-dev.yaml

```

1  # Here is an example file of how to give deployment
2  values for a specific environment
3  # It is necessary to reference this file in the
4  deployment.yaml
5
6  image:
7    ## @param image.registry micro image registry to fill
8    repository: san-narq-ref/darwin-node/sgtnodefastify01
9    tag: development
10   ## @param image.pullPolicy darwin micro image pull
11   policy
12   ## Specify a imagePullPolicy
13   ## Defaults to 'Always' if image tag is 'latest', else
    set to 'IfNotPresent'
    ## ref: https://kubernetes.io/docs/user-
    guide/images/#pre-pulling-images
    ##
    pullPolicy: Always

```

values-pre.yaml

```

1  # Here is an example file of how to give deployment
2  values for a specific environment
3  # It is necessary to reference this file in the
4  deployment.yaml
5
6  image:
7    ## @param image.registry micro image registry to fill
8    repository: san-narq-ref/darwin-node/sgtnodefastify01
9    ## @param image.pullPolicy darwin micro image pull
10   policy
11   ## Specify a imagePullPolicy
12   ## Defaults to 'Always' if image tag is 'latest', else
    set to 'IfNotPresent'

```

```
## ref: https://kubernetes.io/docs/user-  
guide/images/#pre-pulling-images  
##  
pullPolicy: IfNotPresent
```

values-pro.yaml

```
1 # Here is an example file of how to give deployment  
2 values for a specific environment  
3 # It is necessary to reference this file in the  
4 deployment.yaml  
5  
6 image:  
7   ## @param image.registry micro image registry to fill  
8   repository: san-narq-ref/darwin-node/sgtnodefastify01  
9   ## @param image.pullPolicy darwin micro image pull  
10  policy  
11   ## Specify a imagePullPolicy  
12   ## Defaults to 'Always' if image tag is 'latest', else  
   set to 'IfNotPresent'  
   ## ref: https://kubernetes.io/docs/user-  
   guide/images/#pre-pulling-images  
   ##  
   pullPolicy: IfNotPresent
```



Important Helm Configuration parameters



Parameter	Description
image.repository	Name of the project and image that we are going to deploy to Kubernetes with HELM
image.registry	Registry from where we are going to retrieve the image for our deployment
darwin.gitRepo	Url of the microservice repository in GitHub
darwin.region	Region where we are going to deploy the microservice
darwin.technologyVersion	NodeJS version
darwin.version	Version that we are going to deploy of our microservice

Parameter	Description
<code>darwin.config.type</code>	Use to configure the configmaps and secrets. Possible values <code>{"cm", "secret", "cm-secret", "none"}</code>
<code>darwin.config.overrideConfigMapName</code>	Allows to overwrite the name of the ConfigMap that contains the configuration.
<code>darwin.config.overrideSecretName</code>	Allows to overwrite the name of the Secret that contains the configuration.
<code>darwin.i18n</code>	Internationalization feature for the application.
<code>darwin.i18n.enabled</code>	Enable or disable the Internationalization feature.
<code>darwin.i18n.path</code>	Allows to change the path of the i18n files.
<code>darwin.i18n.overrideConfigMapName</code>	Allows to overwrite the name of the ConfigMap that contains the i18n files.



Chart Configuration



For more in-depth information, please visit the Darwin Node Helm chart repository [Darwin NodeJS Helm chart repository](#). There you will find all the information about the parameters that you can configure.

OBSERVABILITY CONFIGURATION

In the `values.yaml` file, you can configure Kafka settings through environment variables specified within the `extraEnvVars` property. This functionality allows you to direct logs either to an ELK stack via Kafka or to the console.:

- **DARWIN_LOGGING_TRANSPORT**: Determines the log transport method. Set to `CONSOLE` to direct logs to the console, or `KAFKA` to send logs to Kafka. The default value is "KAFKA".
- **DARWIN_LOGGING_KAFKA_USERNAME**: Specifies the Kafka username. This is required when `DARWIN_LOGGING_TRANSPORT` is set to `KAFKA`.

- **DARWIN_LOGGING_KAFKA_PASSWORD**: Specifies the Kafka password. This is required when `DARWIN_LOGGING_TRANSPORT` is set to `KAFKA`.

```
extraEnvVars:
- name: DARWIN_LOGGING_TRANSPORT
  value: "KAFKA"
- name: DARWIN_LOGGING_KAFKA_USERNAME
  value: "MY_USERNAME"
- name: DARWIN_LOGGING_KAFKA_PASSWORD
  valueFrom:
    secretKeyRef:
      name: "secret-darwin-observability"
      key: "password"
      optional: false
```



Kafka environment variables



By default, the `DARWIN_LOGGING_TRANSPORT` variable is set to `KAFKA`. If you do not set the `DARWIN_LOGGING_KAFKA_USERNAME` and `DARWIN_LOGGING_KAFKA_PASSWORD` variables, **your microservice will not be deployed**. Setting 'DARWIN_LOGGING_TRANSPORT' to "CONSOLE" will avoid sending the logs to Kafka and send it to the console, and no further configuration will be necessary. The secret must be referenced like the example above.

SECURITY WHITE LIST CONFIGURATION

Other environment variables can also be configured in `extraEnvVars`, such as **DARWIN_SECURITY_WHITE_LIST**. This variable is an array of strings that specifies the microservice routes that do not require security. Note that double quotes (") within the array must be escaped with a backslash (\).

```
extraEnvVars:
- name: DARWIN_SECURITY_WHITE_LIST
  value: "[\"/my/whitelisted/endpoint\"]"
```

Secrets Configuration

There are three types of secrets in Github.com:

- **Organization secrets**: Secrets that can be used by all repositories in the organization.
- **Repository secrets**: Secrets that can be used only by the repository.

- **Environment secrets:** Secrets that can be used only by the repository and the environment.



Setup GitHub Secrets



Secrets are set up, managed and deployed with **the vault**.

Deploy Secrets

Those credentials are referenced in the `Gluon Open Application Model repository` and defined at **environment secrets level**, so we have to create the secrets in each environment (CERT, PRE and PRO).

DEPLOYMENT INFRASTRUCTURE

For accessing to the **OC** Kubernetes Cluster, it is necessary to create authentication secrets for this key:

- `credentialsId`

For accessing to the **EKS** and **AKS** Kubernetes Cluster, or to the **APIGEE** and **IBM** API Managers, it is necessary to create authentication secrets for these keys:

- `credentialUserId`
- `credentialPassId`

REGISTRIES INFRASTRUCTURE

For accessing to the **Harbor**, **Artifactory**, **ECR**, and **ACR** Artifacts Stores, it is necessary to create authentication secrets for these keys:

- `usernameId`
- `passwordId`

For getting more information about the authentication keys, please refer to [Parameters available by type of infrastructure component](#)

Build and Deploy your application

Git Flow

Now we are going to describe the steps that a user has to perform in order to make a complete cycle, from the construction process (CI) to the deployment through the DEV, PRE and PRO environments (CD)

The cycle explained below is based on **GitFlow**

Quality Gates

We want our microservices to have the maximum quality in Gluon prior to deployments to production environments, so it will be necessary to have the OK in:

- **Sonar**: Code Quality and test coverage
- **Fortify**: Analysis of the vulnerabilities of our source code
- **Sonatype**: Analysis of the vulnerabilities of our dependencies.
- **Gluon Testing**: Functional tests (Execution of Functional Test with Gluon Testing framework after that the deployment PRE was successful)



Quality Gates

Without these QG resolved we will only be able to deploy to development environments, being subject to these validations the pre/production environments.

Pull Request from Feature to Develop

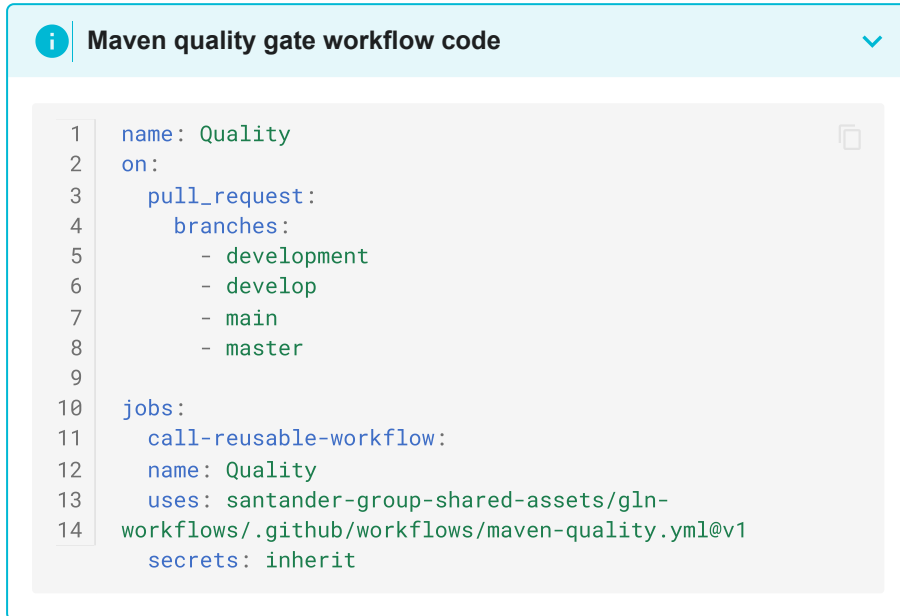
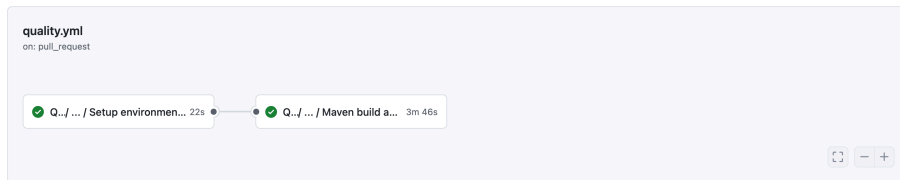
We will start working on the "Feature" branches of our GitHub repository, and we will be integrating our changes into the integration branch (develop or development)

When we create the **Pull Request** event from our **"Feature" branch to the integration branch** (develop or development), the quality.yml (Sonar) and security.yml (Fortify & Sonatype) workflows will be executed automatically.

Next, we detail which steps are executed in each of these two workflows:

QUALITY GATE WORKFLOW

- **Setup environment variables**: Load the properties defined in properties.env file and in the configuration project.
- **Maven build & Sonar scan**: Executes the default commands of MAVEN_BUILD_GOAL: clean verify



SECURITY WORKFLOW

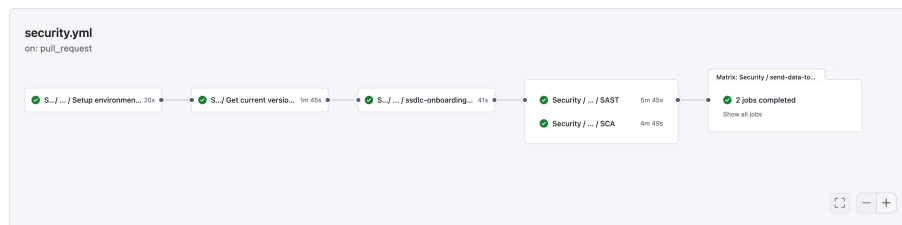
- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project.
- **Get the current version from pom:** Read the pom.xml to get the version of the component.
- **SSDLC Onboarding Check:** Check if the component already exists in Fortify SSC.
- **SAST:** Executes the reusable Fortify SAST workflow to perform a SAST scan with Fortify and validate the quality gates.
- **SCA:** Executes the reusable The Sonatype SCA workflow to perform a Sonatype SCA scan and analyze third party components from a repository.
- **Send information to Elasticsearch:** Send data to elasticsearch related with Fortify analysis.
- **Send information to Elasticsearch:** Send data to elasticsearch related with Sonatype analysis.

i
Maven security workflow code

```

1  name: Security
2  on:
3    pull_request:
4      branches:
5        - development
6        - develop
7        - main
8        - master
9
10 jobs:
11   call-reusable-workflow:
12     name: Security
13     uses: santander-group-shared-assets/gln-
14     workflows/.github/workflows/maven-security-image.yml@v1
       secrets: inherit

```



Push to Develop

When approving the Pull Request of the previous step on the integration branch (development/develop) we will generate a push event on this branch and therefore the ci-gfw.yml workflow will be executed automatically.

Next, we detail which steps are executed in this workflow:

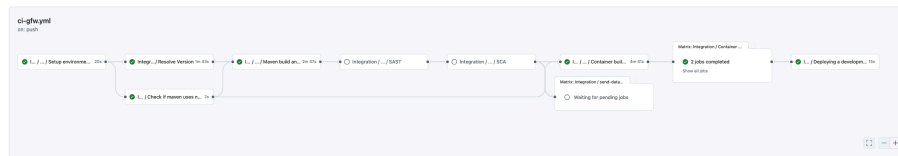
INTEGRATION WORKFLOW

- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project.
- **Resolve version:** Resolve the version of the component.
- **Check if maven uses native build:** Check if it is necessary to configure a native compilation.
- **Maven build and Sonar scan:** Executes the default commands of MAVEN_BUILD_GOAL: clean verify
- **Fortify SAST** Executes the reusable Fortify SAST workflow to perform a SAST scan with Fortify and validate the quality gates.

- **Sonatype SCA:** Executes the reusable The Sonatype SCA workflow to perform a Sonatype SCA scan and analyze third party components from a repository.
- **Send information to Elasticsearch:** Send data to elasticsearch related with SAST and SCA analysis.
- **Container build and push:**
 - It takes the feature or development branch as image version, and sets the environment variable TAG_VERSION with this value.
 - Then execute the command `mvn clean package -Dmaven.test.skip=true` to build the application artifact.
 - It builds the docker image based on Dockerfile located in the project and makes a push to one or several image registries associated with the Kubernetes infrastructures configured in the environment `cd.yaml` file.
 - It executes a **sysdig scan**.
 - Finally, scan the image vulnerabilities and send the data related to the image build to elasticsearch.
- **Send data to elasticsearch** related with the image registries where the image has been published.
- **Deploying a development:** This job call to workflow `maven-cd-image.yaml` for deploy our microservice to cert environment

**Maven CI Image workflow code**

```
1  name: Integration
2    on:
3      push:
4        branches:
5          - development
6          - develop
7          - feature/*
8          - fix/*
9
10   jobs:
11     call-reusable-workflow:
12       name: Integration
13       uses: santander-group-shared-assets/gln-
14 workflows/.github/workflows/maven-ci-gfw.yml@v1
15     with:
16       technology: 'Java_Maven'
17       secrets: inherit
```



If everything works correctly, we will have our image uploaded to the registry, and we will have the information in Sonar and Fortify.

[Amazon ECR](#) >
 [Registro privado](#) >
 [Repositorios](#) >
 gluon-adoption/sgt-gluonad-mcradopcicd

gluon-adoption/sgt-gluonad-mcradopcicd

Ver comandos de envío

Imágenes (6)

Eliminar

Detalles

Analizar

☐	Etiqueta de imagen ▾	Tipo de artefacto	Enviado a ▾	Tamaño (MB) ▾	URI de imagen	Resumir
☐	1.1.0	image	06 de septiembre de 2024, 08:49:26 (UTC+02)	231.39	Copiar URI	sha256:e68985244938c0...
☒	development	image	05 de septiembre de 2024, 16:43:56 (UTC+02)	231.39	Copiar URI	sha256:1e6bc1ef9c84841...

DEPLOY WORKFLOW

To learn how the common deployment workflow works, applicable to all technologies, you can refer to the [Common CD Workflow](#) section of the documentation.

If everything works correctly, we will have our image deployed in the CERT environment.

Amazon Elastic Kubernetes Service

Cústeres

▼ Amazon EKS Anywhere

Suscripciones a Enterprise Events

▼ Servicios relacionados

Amazon ECR

AWS Batch

Configuración de la consola

Documentación

Enviar comentarios

< Información general Recursos Informática Redes Complementos Acceso Observabilidad Información sobre la actualización Historial de actualizaciones

Tipos de recursos

▼ Cargas de trabajo

PodTemplates

Pods

ReplicaSets

Implementaciones

StatefulSets

DaemonSets

Trabajos

Consoles

PriorityClasses

HypervisorBasedInterceptors

Cargas de trabajo: Pods (2)

Ver detalles

Pod es el objeto de Kubernetes más pequeño y sencillo. Un pod representa un conjunto de contenedores en ejecución en su clúster. Más información

gluon-adoption Filtros: Pods por nombre

Nombre	Antigüedad
☐ sgt-gluonad-microdardwinaws-micro-java-54db7f464c-qy96z	Creado 📅 24 de junio de 2024, 13:10 (UTC+02:00)
☐ sgt-gluonad-micronewiced-micro-java-65c7b6e99-gzgjm	Creado 🕒 hace una hora

Pull Request from the development branch to the main branch

When we are ready to promote our microservice to the PRE/PRO environments, we will create a Pull Request from the integration branch (develop or development) to the main branch.

This event will launch the **version-validation.yml workflow**, **quality.yml workflow** (Sonar) and **security.yml workflows** (Fortify & Sonatype).

Push to the main branch

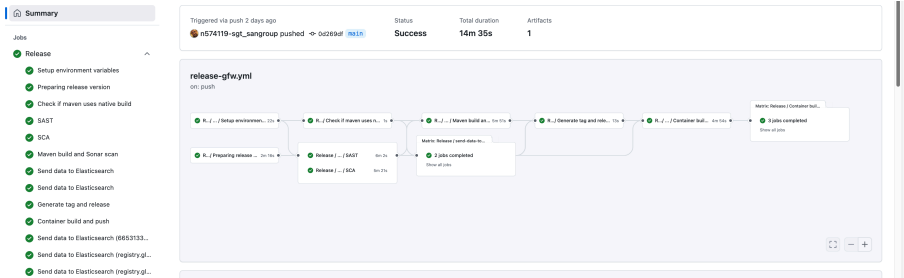
When we approve the PR of the previous step on the main branch, we will generate a push event on this branch, and therefore the `release-gfw.yml` workflow will be executed automatically.

RELEASE WORKFLOW

- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project. The configuration project is obtained according to the secrets defined here.
- **Preparing release version:** Update version pom in the branch (main or master) with the Release version. Get last commit in the branch.
- **Check if maven uses native build:** Check if it is necessary to configure a native compilation.
- **SAST** Executes the reusable Fortify SAST workflow to perform a SAST scan with Fortify and validate the quality gates.
- **SCA:** Executes the reusable The Sonatype SCA workflow to perform a Sonatype SCA scan and analyze third party components from a repository.
- **Maven build & Sonar scan:** Executes the default commands of MAVEN_BUILD_GOAL: clean verify. If you want to overwrite the maven command you can configure in properties.env
- **Send information to Elasticsearch:**Send data to elasticsearch related with Fortify analysis.
- **Send information to Elasticsearch:**Send data to elasticsearch related with Sonatype analysis.
- **Generate tag and release:** Create the release tag and generate the GitHub Release.
- **Container build and push:**
 - It takes the version from the resolve version job to generate the release identifier as the image version, and sets the environment variable TAG_VERSION with this value.
 - After that, it generates the application and the distribution of the configuration, to continue,
 - It builds the docker image based on Dockerfile located in the project and make a push to one or several image registries associated with the Kubernetes infrastructures configured in the environment `cd.yaml` file.
 - The container images are pushed with the calculated release version tag.
 - It executes a **sysdig scan**.
 - Finally, scan the image vulnerabilities and send the data related to the image build to elasticsearch.
- **Send data to elasticsearch** related to the image.

Release workflow code

```
1  name: Release
2  on:
3    push:
4      branches:
5        - main
6        - master
7  jobs:
8    call-reusable-workflow:
9      name: Release
10     uses: santander-group-shared-assets/gln-
11     workflows/.github/workflows/maven-release-gfw.yml@v1
12   with:
13     technology: 'Java_Maven'
14     secrets: inherit
```



At the end of the workflow, we can see a GitHub Release has been created using the GitHub Tag generated with the Release workflow.

ReleasesTags

Find a release

2 days ago

gluon-app

1.0.0

0d269df

Compare

v1.0.0Latest

Published release v1.0.0 and tag 1.0.0

Assets2

Source code (zip)2 days ago

Source code (tar.gz)2 days ago

And a new container image has been pushed to the container registry using the same name as the tag of the version:

Amazon ECR > Registro privado > Repositorios > gluon-adoption/sgt-gluonad-mcradopcidd

gluon-adoption/sgt-gluonad-mcradopcidd

Ver comandos de envio

Imágenes (6)

EliminarDetallesAnalizar

Buscar artefactos

<1>

Etiqueta de imagen	Tipo de artefacto	Enviado a	Tamaño (MB)	URI de imagen	Resumir
1.1.0	Image	06 de septiembre de 2024, 08:49:26 (UTC+02)	231.39	Copiar URI	sha256:e6895244938c0...
development	Image	05 de septiembre de 2024, 16:43:56 (UTC+02)	231.39	Copiar URI	sha256:1e6bc1e9c84841...

PRE AND PRO DEPLOYMENT

The **Deploy Workflow** can be called manually to deploy the microservice in the PRE and PRO environments.

To learn how the common deployment workflow works, applicable to all technologies, you can refer to the **Common CD Workflow** section of the documentation.

Retry a deploy

RETRY A DEPLOYMENT EXECUTION

If the **"Container build and push"** job runs successfully and publishes the Release version, you can rerun the deployment workflow.

Deploy using the Gluon Release Management

Gluon Release Management is a Gluon portal integration process that aims to automate the creation of releases and tasks in Servicenow ITSM, as well as the orchestration of deployment in Github through the portal itself.

For getting to know how to deploy using this tool, please refer to the **Release Management** documentation.

Trunk Based Development

Now we are going to describe the steps that a user has to perform in order to make a complete cycle, from the construction process (CI) to the deployment through the DEV, PRE and PRO environments (CD)

The cycle explained below is based on **Trunk Based Development**.

Quality Gates

We want our microservices to have the maximum quality in Gluon prior to deployments to production environments, so it will be necessary to have the OK in:

- **Sonar**: Code Quality and test coverage
- **Fortify**: Analysis of the vulnerabilities of our source code
- **Sonatype**: Analysis of the vulnerabilities of our dependencies.
- **Gluon Testing**: Functional tests (Execution of Functional Test with Gluon Testing frameworks after the deployment PRE is successful)



Quality Gates

Without these QG resolved we will only be able to deploy to development environments, being subject to these validations the pre/production environments.

Pull Request from Feature to Main

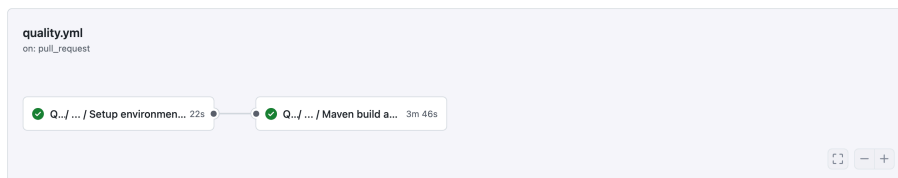
We will start working on the "Feature" branches of our GitHub repository, and we will be integrating our changes into the main branch.

When we create the **Pull Request** event from our **Feature** branch to the main branch, the `quality.yml` (Sonar), `security.yml` (Fortify & Sonatype), `version-validation.yml` (Version Release validation) and `archunit.yml` (Darwin Code Analysis Plugin) workflows will be executed automatically.

Next, we detail which steps are executed in each of these two workflows:

QUALITY GATE WORKFLOW

- **Setup environment variables:** Load the properties defined in `properties.env` file and in the configuration project.
- **npm build & Sonar scan:** Executes the default commands of `npm_BUILD_GOAL: clean verify`



 npm quality gate workflow code

```
1  name: Quality
2
3  on:
4    pull_request:
5      branches:
6        - development
7        - develop
8        - main
9        - master
10       - release-v[0-9]+.[0-9]+.[xX]
11
12  jobs:
13    call-reusable-workflow:
14      name: Quality
15      uses: santander-group-shared-assets/gln-
16      workflows/.github/workflows/npm-quality-image.yml@v1
17      secrets: inherit
```

SECURITY WORKFLOW

- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project.
- **Get the current version from pom:** Read the pom.xml to get the version of the component.
- **SSDLC Onboarding Check:** Check if the component already exists in Fortify SSC.
- **SAST:** Executes the reusable Fortify SAST workflow to perform a SAST scan with Fortify and validate the quality gates.
- **SCA:** Executes the reusable The Sonatype SCA workflow to perform a Sonatype SCA scan and analyze third party components from a repository.
- **Send information to Elasticsearch:** Send data to elasticsearch related with Fortify analysis.
- **Send information to Elasticsearch:** Send data to elasticsearch related with Sonatype analysis.
- **Container build and push:**
 - It takes the feature branch as image version, and sets the environment variable TAG_VERSION with this value.
 - It builds the docker image based on Dockerfile located in the project and makes a push to one or several image registries associated with the Kubernetes infrastructures configured in the environment cd.yaml file.

- It executes a **sysdig scan**.
- Finally, scan the image vulnerabilities and send the data related to the image build to elasticsearch.

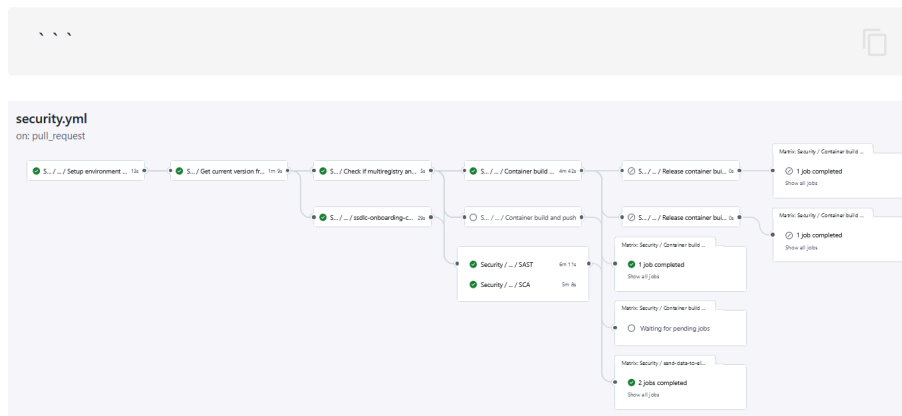
npm security workflow code

```
``yaml
linenums="1"
```

name: Security

on: pull_request: branches: - development - develop - main - master - release-v[0-9]+.[0-9]+.[xX]

jobs: call-reusable-workflow: name: Security uses: santander-group-shared-assets/gln-workflows/.github/workflows/npm-security-image.yml@v1 secrets: inherit



Push to Main

When approving the Pull Request of the previous step on the main branch we will generate a push event on this branch and therefore the ci-tbd.yml workflow will be executed automatically.

Next, we detail which steps are executed in this workflow:

INTEGRATION WORKFLOW

- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project.
- **Resolve version:** Resolve the version of the component.
- **Check if npm uses native build:** Check if it is necessary to configure a native compilation.

- **npm build and Sonar scan:** Executes the default commands of npm_BUILD_GOAL: clean verify
- **Container build and push:**
 - It takes the feature or development branch as image version, and sets the environment variable TAG_VERSION with this value.
 - It builds the docker image based on Dockerfile located in the project and makes a push to one or several image registries associated with the Kubernetes infrastructures configured in the environment `cd.yaml` file.
 - It executes a **sysdig scan**.
 - Finally, scan the image vulnerabilities and send the data related to the image build to elasticsearch.
- **Send data to elasticsearch** related with the image registries where the image has been published.
- **Create tag and Pre release** Create a tag with the version of the component and generate a pre-release in GitHub.
- **Deploying a development:** This job call to workflow npm-cd-image.yaml for deploy our microservice to cert environment



npm CI Image workflow code

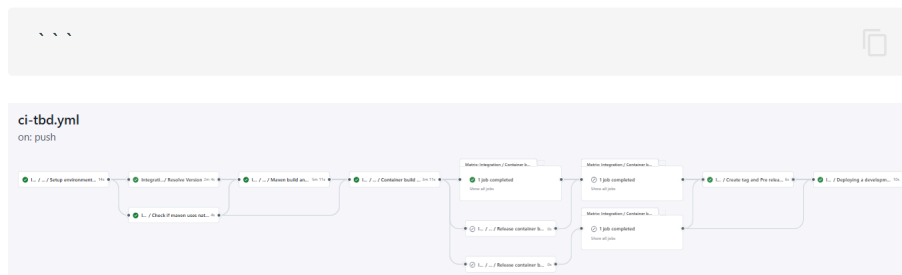


```
``yaml linenums="1"
```

name: Integration

on: push: branches: - main - master - release-v[0-9]+.[0-9]+.[xX]

jobs: call-reusable-workflow: name: Integration uses: santander-group-shared-assets/gln-workflows/.github/workflows/npm-ci-tbd.yml@v1 with: technology: 'npm' secrets: inherit



If everything works correctly, we will have our image uploaded to the registry, and we will have the information in Sonar and Fortify.

Amazon ECR > Registro privado > Repositorios > gluon-adoption/sgt-gluonad-mcradopcid

gluon-adoption/sgt-gluonad-mcradopcid Ver comandos de envío

Imágenes (6) Eliminar Detalles Analizar

☐	Etiqueta de imagen	Tipo de artefacto	Enviado a	Tamaño (MB)	URI de imagen	Resumir
☐	1.1.0	Image	06 de septiembre de 2024, 08:49:26 (UTC+02)	231.39	Copiar URI	sha256:e68985244938c0...
☐	development	Image	05 de septiembre de 2024, 16:43:56 (UTC+02)	231.39	Copiar URI	sha256:1e6bc1e9c84841...

DEPLOY WORKFLOW

To learn how the common deployment workflow works, applicable to all technologies, you can refer to the **Common CD Workflow** section of the documentation.

If everything works correctly, we will have our image deployed in the CERT environment.

Amazon Elastic Kubernetes Service

Clústeres

- Amazon EKS Anywhere
- Suscripciones a Enterprise
- Amazon ECR
- AWS Batch

Configuración de la consola

Documentación

Enviar comentarios

Recursos

Informática

Redes

Complementos

Acceso

Observabilidad

Información sobre la actualización

Historial de actualiz.

Tipos de recursos

- Cargas de trabajo
- PodTemplates
- Pods
- ReplicaSets
- Implementaciones
- StatefulSets
- DaemonSets
- Trabajos
- CronJobs
- PriorityClasses
- HorizontalPodAutoscalers

Cargas de trabajo: Pods (2)

Pod es el objeto de Kubernetes más pequeño y sencillo. Un pod representa un conjunto de contenedores en ejecución en su clúster. [Más información](#)

gluon-adoption

Nombre	Antigüedad
sgt-gluonad-microdarwinaws-micro-java-54db7f464c-q962	Creado 24 de junio de 2024, 13:10 (UTC+02:00)
sgt-gluonad-micronewctid-micro-java-65c7cb6d99-zg9m	Creado hace una hora

Release

When we want to publish a final version, we will do it from the GitHub releases page. First, we select the previously created version (pre-release or draft version) and then, uncheck the "Set as pre-release" option and click "Publish release". The release-tbd.yml workflow will run automatically.

RELEASE WORKFLOW

- **Setup environment variables:** Load the properties defined in properties.env file and in the configuration project. The configuration project is obtained according to the secrets defined here.
- **Getting pre-release id:** Get pre-release version, update version pom in the branch (main or master) with the Release version. Get last commit in the branch.
- **Check if npm uses native build:** Check if it is necessary to configure a native compilation.
- **npm build & Sonar scan:** Executes the default commands of npm_BUILD_GOAL: clean verify. If you want to overwrite the npm command you can configure in properties.env.
- **SSDLC Onboarding Check:** Check if the component already exists in Fortify SSC.

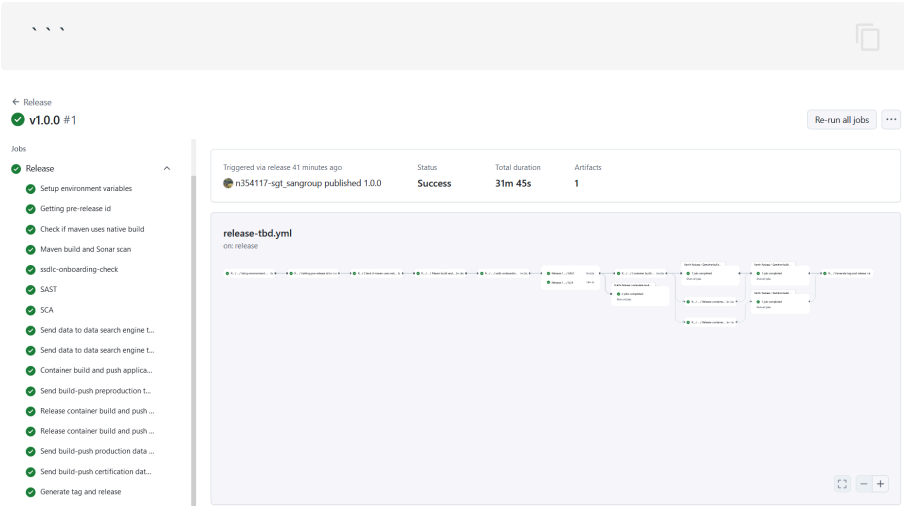
- **SAST** Executes the reusable Fortify SAST workflow to perform a SAST scan with Fortify and validate the quality gates.
- **SCA**: Executes the reusable The Sonatype SCA workflow to perform a Sonatype SCA scan and analyze third party components from a repository.
- **Send information to Elasticsearch**: Send data to elasticsearch related with Fortify analysis.
- **Send information to Elasticsearch**: Send data to elasticsearch related with Sonatype analysis.
- **Container build and push**:
 - It takes the version from the resolve version job to generate the release identifier as the image version, and sets the environment variable TAG_VERSION with this value.
 - After that, it generates the application and the distribution of the configuration, to continue,
 - It builds the docker image based on Dockerfile located in the project and make a push to one or several image registries associated with the Kubernetes infrastructures configured in the environment `cd.yaml` file.
 - The container images are pushed with the calculated release version tag.
 - It executes a **sysdig scan**.
 - Finally, scan the image vulnerabilities and send the data related to the image build to elasticsearch.
- **Send data to elasticsearch** related to the image.
- **Release container build and push application to production**: Generate tag and push them to the artifact store.
- **Release container build and push application to certification**: Generate tag and push them to the artifact store.
- **Generate tag and release**: Create the release tag and generate the GitHub Release.

**Release workflow code**

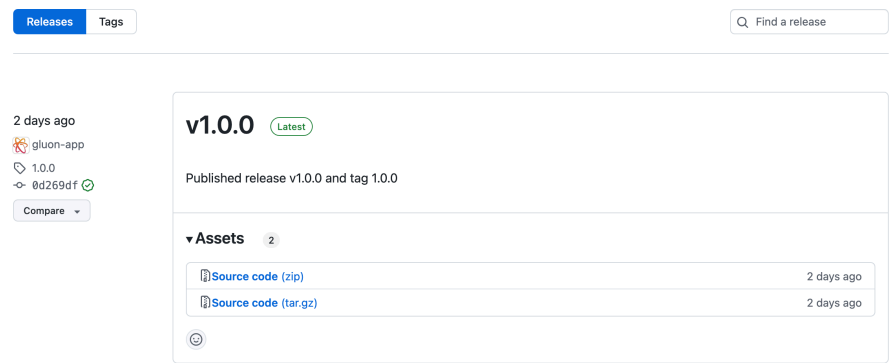
```
```yaml linenums="1"
```

name: Release

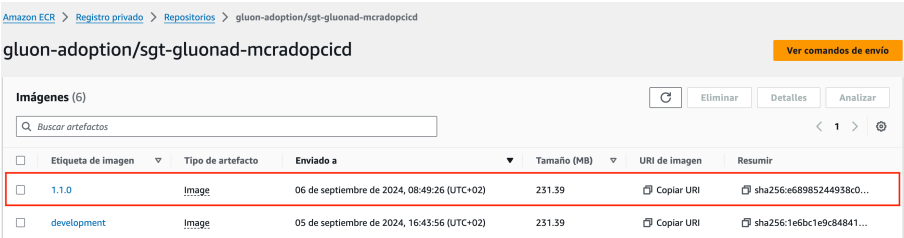
on: release: types: - published jobs: call-reusable-workflow: name: Release  
uses: santander-group-shared-assets/gln-workflows/.github/workflows/npm-release-tbd.yml@v1 with: technology: 'npm' secrets: inherit



At the end of the workflow, we can see a GitHub Release has been created using the GitHub Tag generated with the Release workflow.



And a new container image has been pushed to the container registry using the same name as the tag of the version:



PRE AND PRO DEPLOYMENT

The **Deploy Workflow** can be called manually to deploy the microservice in the PRE and PRO environments.

To learn how the common deployment workflow works, applicable to all technologies, you can refer to the [Common CD Workflow](#) section of the documentation.

## Retry a deploy

### RETRY A DEPLOYMENT EXECUTION

If the **"Container build and push"** job runs successfully and publishes the Release version, you can rerun the deployment workflow.

## Deploy using the Gluon Release Management

Gluon Release Management is a Gluon portal integration process that aims to automate the creation of releases and tasks in Servicenow ITSM, as well as the orchestration of deployment in Github through the portal itself.

For getting to know how to deploy using this tool, please refer to the [Release Management](#) documentation.

## Fix/Release Flow

We are now going to describe the **Fix/Release (FR)** workflow, which allows us to manage incidents and updates in production environments while development remains active in other branches. This flow enables quick fixes and efficient deployment of new versions without affecting regular development cycles.

The **FR** workflow complements existing workflows such as **GitFlow** or **Trunk-Based Development**, providing a specific process to handle critical incidents and updates.

### General Overview of the Flow

The **Fix/Release (FR)** flow behaves the same in both **Trunk-Based Development (TBD)** and **GitFlow Workflow (GFW)** strategies, even though it is implemented differently in each:

#### In Trunk-Based Development (TBD)

In TBD, no new specific user workflows have been created. Instead, the `ci-tbd.yml` user workflow has been updated to also run on branches following the pattern `release-v[0-9]+.[0-9]+.[xX]`. This allows the FR workflow to be managed in TBD without additional workflows.



`release-vX.X.x` branch (in this case, `release-v1.0.x`) starting from the specified tag `1.0.1`, from which fixes will be managed.

## 2. Create and Merge `fix/` Branches:

- For each fix, create a `fix/[TASK]` branch and open a **Pull Request (PR)** to `release-vX.X.x`.
- **Automatic Workflow Execution:** When the PR is created, the associated workflows for **version verification**, **quality checks**, and **security validations** will automatically run.
- Once the PR is merged into `release-vX.X.x`, the `ci-fix-gfw.yml` or `ci-tbd.yml` workflow (depending on the strategy) will be triggered, generating a draft pre-release in GitHub and deploying the version to the certification environment.

## 3. Publish the Pre-Release:

- After validating the pre-release, publish it. This will trigger the `release-fix-gfw.yml` or `release-tbd.yml` workflow, managing the release in pre-production or production environments.

# Changelog

## Version 1.1.1

- Migrate to the latest component template model (global).
- Update the component template microservice scaffolding to use the new CI/CD framework components.